Marco Cartago

Alessandro Longato

Tonet Lorenzo

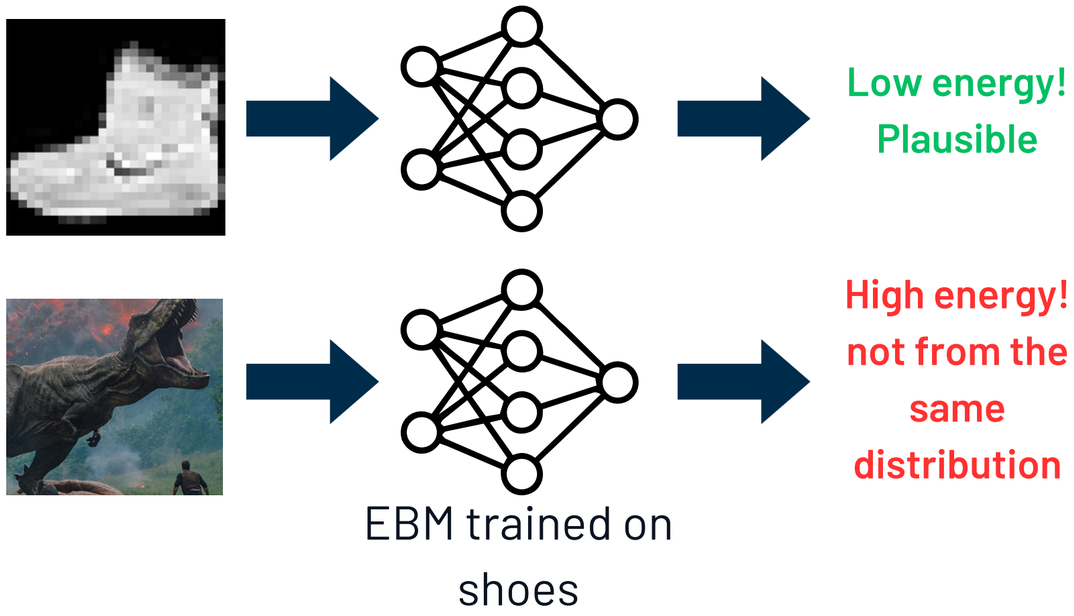
Energy Based Density Factorization

Generative task and EBMs

The generative task is to learn a probability distribution parametrized by θ and formalized as

$$p(x) = \frac{e^{-\mathrm{E}_\theta(x)}}{Z_\theta}$$

over a dataset of images where typically $E_\theta(x)$ is a neural network.



Energy-Based Models are a flexible class of generative models that learn an unnormalized energy function, a scalar value scores to data points, where lower energy corresponds to higher probability.

Training of the model

Contrastive Divergence (CD) is an approximate Maximum-Likelihood learning algorithm proposed by Geoffrey Hinton initially with the purpose of learning Markov Random Fields.

$$\mathcal{L}_{\mathrm{CD}} = \mathbb{E}_{x^+ \sim p_{data}}[\mathbf{E}_\theta(x^+)] - \underbrace{\mathbb{E}_{x^- \sim p_\theta}[\mathbf{E}_\theta(x^-)]}_{\text{Requires sampling}}$$

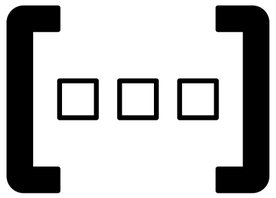
Minimizing the CD loss is equivalent to maximize the log-likelihood!

How do you sample?

To sample from an high dimensional domain such image-space we employ **Stochastic Gradient Langevin Dynamics** (SLGD). The update rule is defined as

$$x_{t+1} = x_t - \gamma \nabla_x \mathrm{E}_\theta(x_t) + \epsilon \qquad \epsilon \sim \mathcal{N}(0, \alpha)$$

Starting from noise and doing a fixed number of iteration we obtain a sample from a "low-energy area" (stationary distribution of the current model)

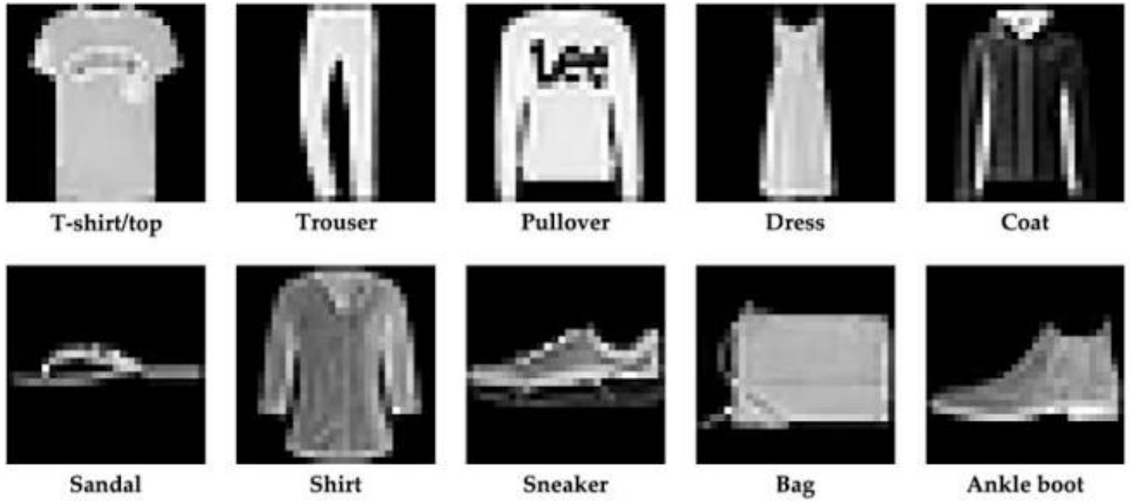


A buffer is used to store past samples, initializing each new chain from a stored sample rather than from noise, so that chains persist and better approximate $p_\theta(x)$

Persistent Contrastive divergence

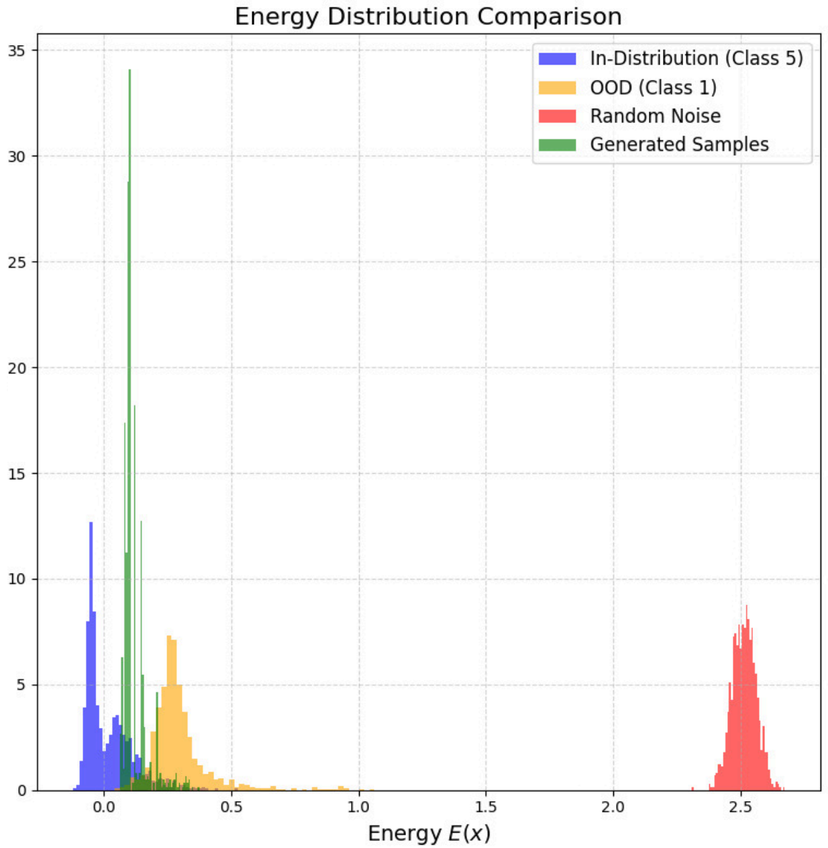
Datasets

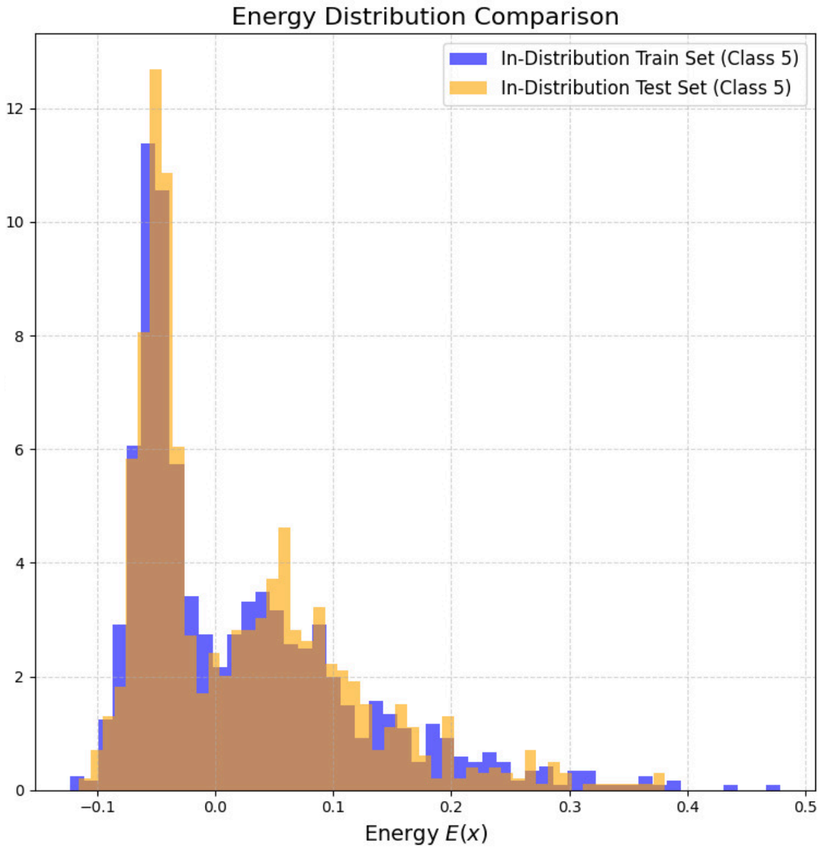
Fashion MNIST



Olivetti faces

Energy Landscape





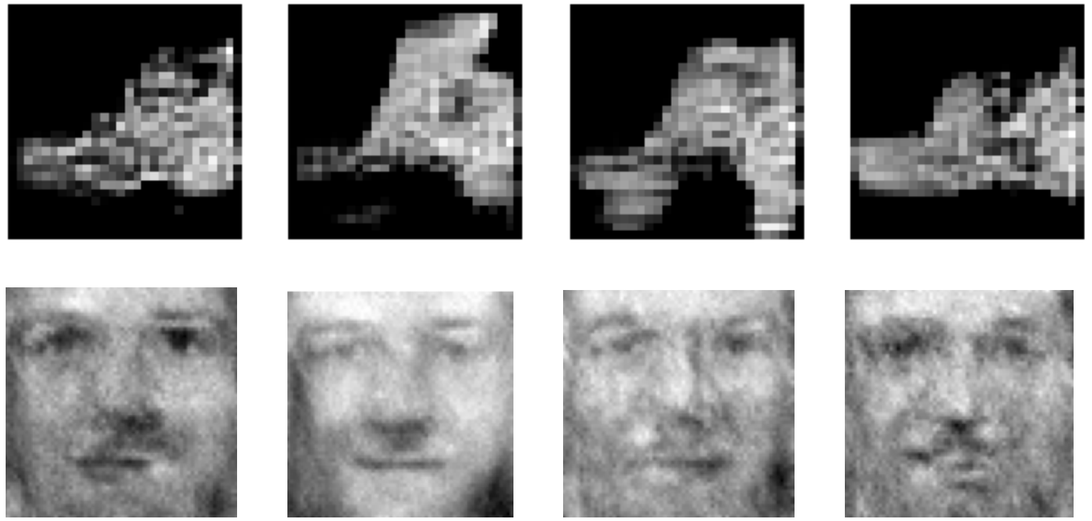
The model learns to assign lower energy to sample from the training set, higher energy to other plausible images and high energy to noise

We should expect that generated samples should have just higher energy than data points

From energy values we can also see that the model isn't overfitting because training and test sets have the same distribution of energy levels

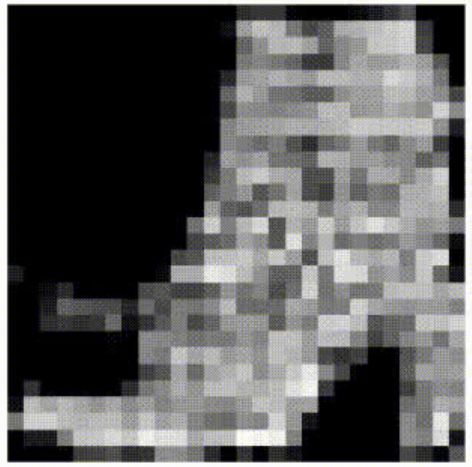
Some quality metrics

Model	FID Olivetti ↑	FID FMNIST ↓
EBM (ours)	**420.67**	185.35
Baseline VAE	430.47	186.25
Base U-net Diffusion	420.92	**111.22**

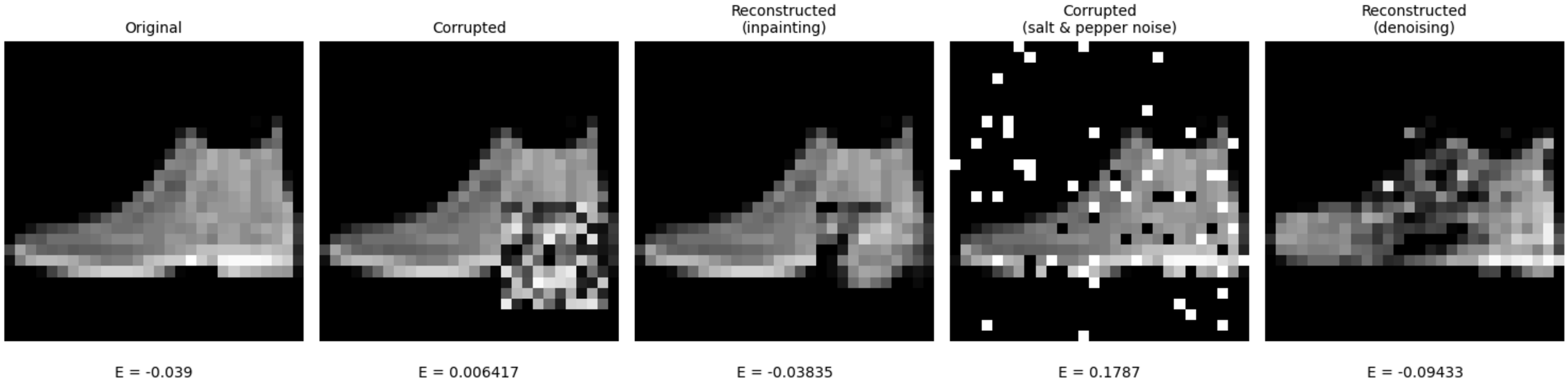


* FID works by performing 3 steps:
  1. Use a **pre-trained feature extractor** to create embeddings of images
  2. Assuming that those feature across the two set are distributed as multivariate Gaussians, **estimate mean and variance of both sets**
  3. Calculate **Frechét distance** as

$$\mathrm{FID} = \|\mu_g - \mu_{\mathcal{D}}\|^2 + \mathrm{Tr}\left(\Sigma_g + \Sigma_{\mathcal{D}} + -2\sqrt{\Sigma_g \Sigma_{\mathcal{D}}}\right)$$



Generalization properties

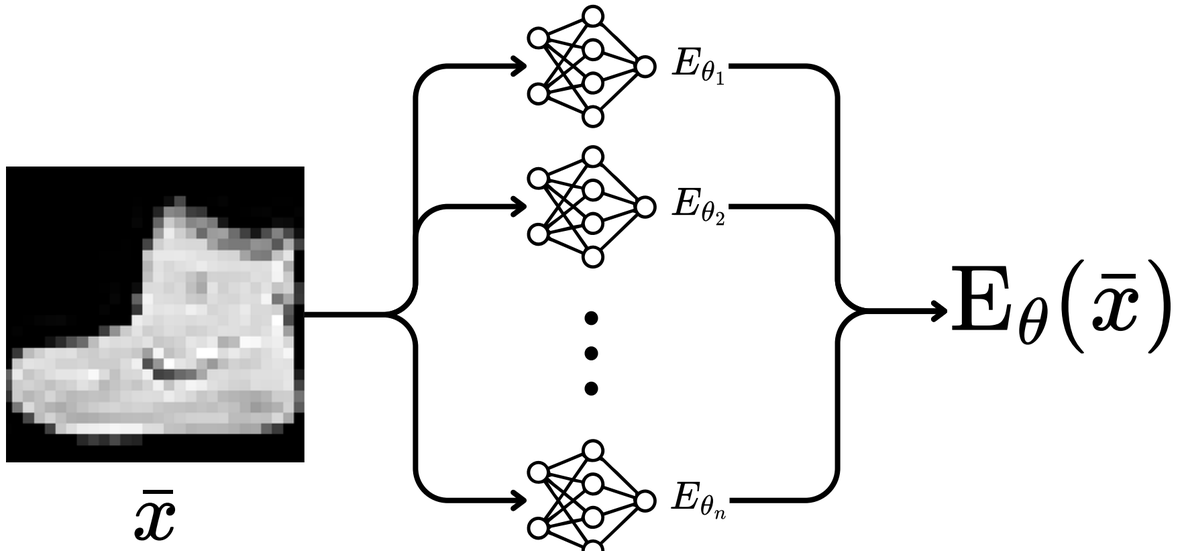


Since the model learns a general energy landscape over the image manifold rather than a direct mapping from noise to data, the same model can be repurposed for reconstruction tasks

Our hypotesis

By exploiting the composition properties of EBMs we can learn a joint probability distribution from data formalized as a factorization of $p_\theta(x)$ made by different energy functions that capture different "semantic concepts" of the distribution.

$$p(x) = p_{\theta_1}(x)p_{\theta_2}(x)\dots p_{\theta_n}(x) = \frac{e^{-(\mathrm{E}_{\theta_1}(x)+\mathrm{E}_{\theta_2}(x)+\dots+\mathrm{E}_{\theta_n}(x))}}{Z_{\theta_1}Z_{\theta_2}\dots Z_{\theta_n}}$$



We will refer to this as "***Energy Head***"

How to specialize heads?

To learn a factorization of the entire energy funcion, we considere the same loss as before given by the Contrastive Divergence framework but we impose a penalization on a certain measure of "correlation" between the output of the heads.

$$\mathcal{L} = \mathcal{L}_{\mathrm{CD}} + \mathrm{Reg}_{\mathcal{D}}(\theta) + \textbf{?}$$

- Linear correlation
- Gradient correlation
- Total Correlation

Since exact TC computation requires evaluating the partition function, we use the approximation proposed in MINE*

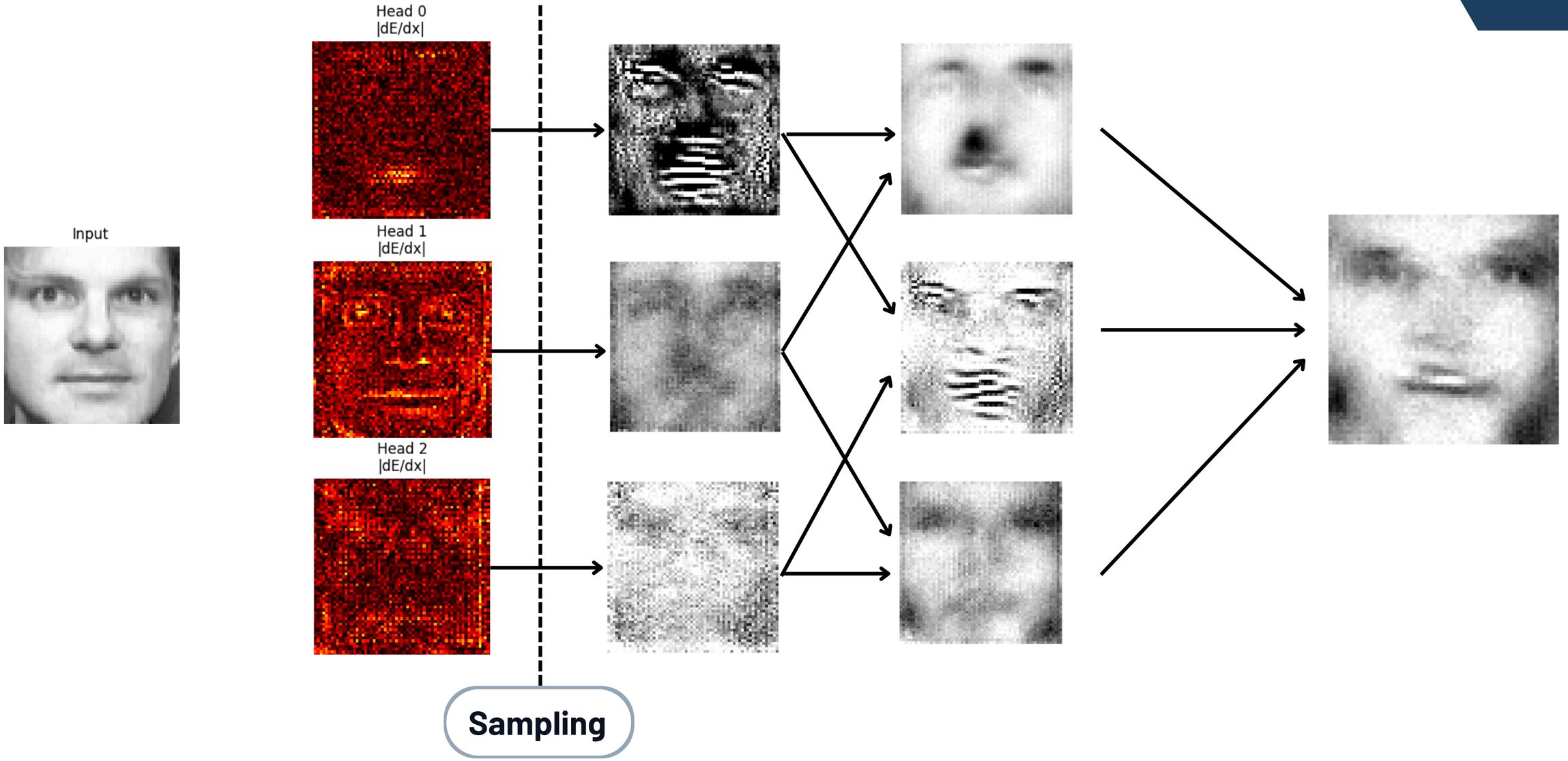
The model learned to delegate **all the work to just one of the heads**, letting the other produce near zero output

Penalysing the gradient in respect to the input data the result was that different heads "looked" to the **same area of the image but different pixels**

Total Correlation between heads performed best, preventing head collapse and helping complementary factorization.

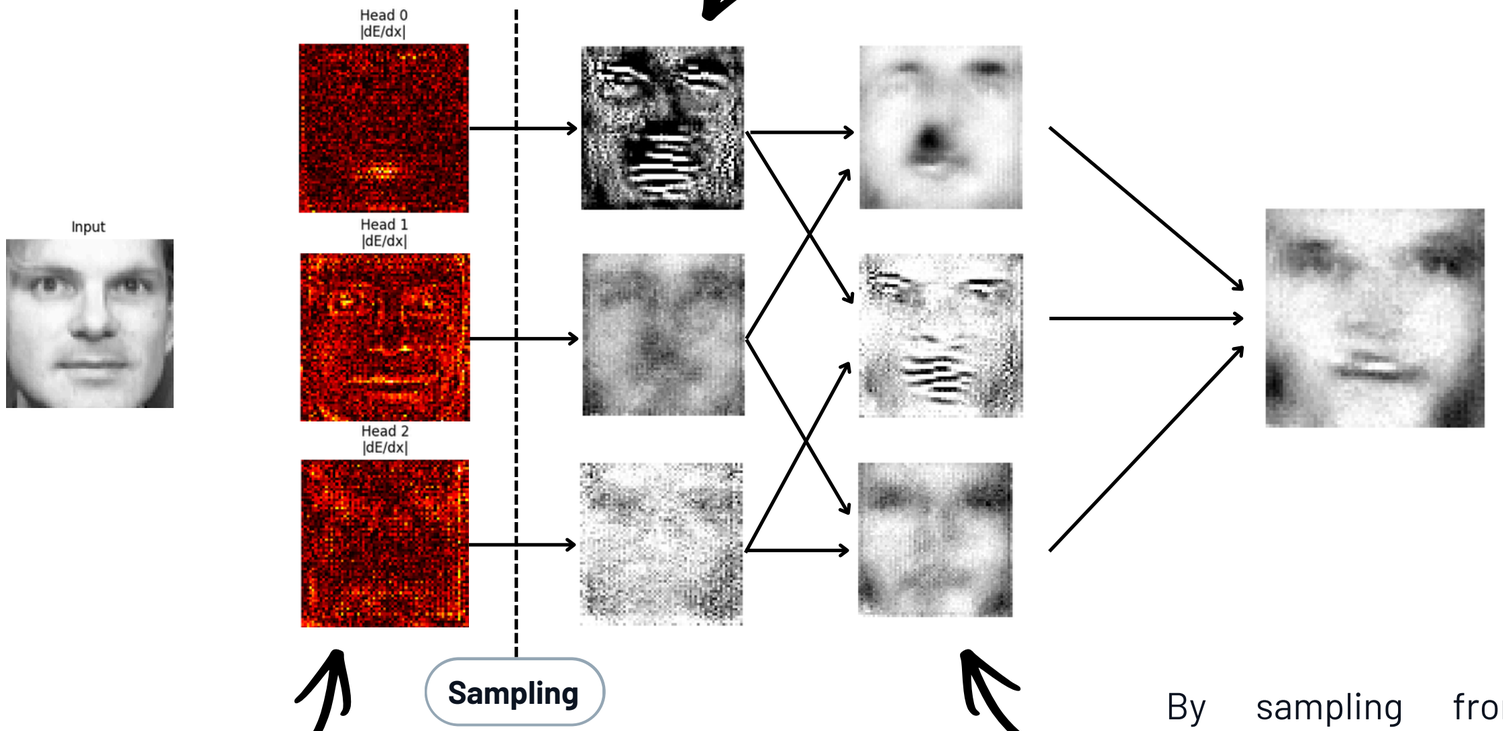
Results ------>

Generation from heads



Generation from heads

By penalizing the TC between energy heads, an emergent behavior is the rise of **sensibility to a specific semantic feature** of the image



Sampling from the probability distribution described by the **single energy head**

By sampling from the probability distribution described by the **combination of two energy heads** it's possible to see both the features described by them

Related works

Implicit Generation and Modeling with Energy-Based Models

Mutual Information Neural Estimation

Mohamed Ishmael Belghazi [1] **Aristide Baratin** [1 2] **Sai Rajeswar** [1] **Sherjil Ozair** [1] **Yoshua Bengio** [1 3 4]
Aaron Courville [1 3] **R Devon Hjelm** [1 4]

Abstract

We argue that the estimation of mutual information between high dimensional continuous random variables can be achieved by gradient descent over neural networks. We present a Mutual Information Neural Estimator (MINE) that is linearly

trast to correlation, mutual information captures non-linear statistical dependencies between variables, and thus can act as a measure of true dependence (Kinney & Atwal, 2014).

Despite being a pivotal quantity across data science, mutual information has historically been difficult to compute (Paninski, 2003). Exact computation is only tractable for discrete

Unsupervised Learning of Compositional Energy Concepts

Yilun Du
MIT CSAIL
yilundu@mit.edu

Shuang Li
MIT CSAIL
lishuang@mit.edu

Yash Sharma
University of Tübingen
yash.sharma@uni-tuebingen.de

Joshua B. Tenenbaum
MIT CSAIL, BCS, CBMM
jbt@mit.edu

Igor Mordatch
Google Brain
imordatch@google.com

https://github.com/marco-cartago/EbmFcT

Thats's all

Bibliography:

- Implicit Generation and Modeling with Energy-Based Models [*Y. Du et al*]
- Unsupervised Learning of Compositional Energy Concepts [*Y. Du et al*]
- Mutual Information Neural Estimation [*M. I. Belghazi et al*]
- Training Restricted Boltzmann Machines using Approximations to the Likelihood Gradient [*T. Tieleman*]
- and others...

Appendix

Contrastive Divergence in *detail*

Objective: maximize likelihood or minimize the negative log-likelihood

$$p(\mathcal{D};\theta) = \prod_{i}^{|\mathcal{D}|} \frac{f(x_i;\theta)}{Z_\theta}$$

$$H(\mathcal{D};\theta) = |\mathcal{D}| log(Z_\theta) - \sum_{i}^{|\mathcal{D}|} log(f(x_i;\theta))$$

Working with the NLL is easier. To simplify computations we divide all by $|\mathcal{D}|$ (division by a scalar doesn't change the argmax)

$$log(Z_\theta) - \frac{1}{D} \sum_{i}^{|\mathcal{D}|} log(f(x_i;\theta))$$

Now our objective is to optimize this function, but there is a problem. Due to the intractability of the partition function, this expression is **generally intractable** so we rely on gradient based optimization. The next step is to compute the gradient of this expression in respect to parameters.

Contrastive Divergence in *detail*

Take the gradient of the previous expression

$$\frac{\partial}{\partial\theta}\log(Z_\theta) - \frac{1}{D}\sum_i^{|\mathcal{D}|}\frac{\partial}{\partial\theta}\log f(x_i;\theta)$$

$$= \frac{\partial}{\partial\theta}\log(Z_\theta) - \left\langle \frac{\partial}{\partial\theta}\log f(x_i;\theta)\right\rangle_D$$

$$* = \underbrace{\left\langle \frac{\partial}{\partial\theta}\log(x_i;\theta)\right\rangle_{p(\mathcal{D};\theta)}}_{\text{Sampling}} - \underbrace{\left\langle \frac{\partial}{\partial\theta}\log f(x_i;\theta)\right\rangle_D}_{\text{Dataset Batch}}$$

Contrastive Divergence

$*$ *Proof:* $\frac{\partial}{\partial\theta}\log(Z_\theta) = \frac{1}{Z_\theta}\frac{\partial}{\partial\theta}Z_\theta = \frac{1}{Z_\theta}\int\frac{\partial}{\partial\theta}f(x)dx = \frac{1}{Z_\theta}\int f(x)\frac{\partial\log f(x)}{\partial\theta}dx = \int\frac{f(x)}{Z_\theta}\frac{\partial\log f(x)}{\partial\theta}dx = \left\langle\frac{\partial}{\partial\theta}\log f(x_i;\theta)\right\rangle_{p(\mathcal{D};\theta)}$ ■

MINE in *detail*

MINE Is a tecnique used to estimate the entropy of a variable trough the use of the *Donsker-Varadhan* equality. It holds for a variable **Z** with distribution **p** that given another reference distribution **q** :

$$\mathrm{H}(Z) = \mathbb{E}_q[-\log q(z)] - \mathrm{D}_{\mathrm{KL}}[q\|p]$$

$$\mathrm{D}_{\mathrm{KL}}[q\|p] = \sup_{f:Z\to\mathbb{R}} \{\mathbb{E}_p[f(z)] - \log \mathbb{E}_q[e^{f(z)}]\}$$

The supremum is replaced with a maximization over a parametrized set of functions f_θ

Algorithm 1 Iutual Information Neural Estimation Algorithm (MINE) [1

1: $\theta \leftarrow$ Network parameters initialization.
2: **repeat**
3: Draw mini-batch of samples: $(X_1, Y_1), (X_2, Y_2), ..., (X_m, Y_m) \sim P_{XY}$.
4: Randomly shuffle $Y_1, Y_2, ..., Y_m$ to generate $\widetilde{Y_1}, \widetilde{Y_2}, ..., \widetilde{Y_m}$
5: Draw m samples from the marginal distribution: $\widetilde{Y_1}, \widetilde{Y_2}, ..., \widetilde{Y_m} \sim P_Y$.
6: Evaluate: $\hat{I}_\theta(X;Y) \leftarrow \frac{1}{m}\sum_{i=1}^m T_\theta(X_i, Y_i) - \log(\frac{1}{m}\sum_{i=1}^m e^{T_\theta(X_i, \widetilde{Y_i})})$.
7: Update network parameters: $\theta \leftarrow \theta + \nabla_\theta \hat{I}_\theta(X;Y)$.
8: **until** convergence

Total Correlation in *"detail"*

Total correlation can be thought as a way as to generalize the concept of mutual information:

$$\mathrm{TC}(X) = -\mathrm{H}(X) + \sum_i \mathrm{H_i}(X_i)$$

We use an approximation inspired by **MINE** used to estimate the total correlation of a variable trough the use of the *Donsker-Varadhan* bound. It holds for a variable **x** with distribution **p** where **pi** are the respective marginals that:

define $\hat{\mathrm{TC}}_\theta(S, S_p)$

1. $B \leftarrow$ Number of samples in X
2. **return** $\frac{1}{B}\sum_{i=1}^{B} f_\theta((S)_i) - \log\left(\frac{1}{B}\sum_{i=1}^{B} e^{f_\theta((S_P)_i)}\right)$

end

$\theta \leftarrow$ Initialize parameters of f
$S \leftarrow$ Matrix $B \times n$
loop

1. Sample B vectors from $p(X_1, X_2, \ldots X_n)$ into S.
2. Shuffle the columns of S to generate S_p
3. Evaluate $\hat{\mathrm{TC}}_\theta(S, S_p)$
4. Update $\theta \leftarrow \theta + \nabla_\theta\left[\hat{\mathrm{TC}}_\theta(S, S_p)\right]$

end

$$\begin{aligned}\mathrm{TC}(x) &= \mathrm{D_{KL}}[p(x_1, x_2, \ldots x_n) \| p(x_1)p(x_2)\cdots p(x_n)] \\ &\approx \max_{\theta\in\Theta}\left\{\mathbb{E}_p[f_\theta(z)] - \log \mathbb{E}_q\left[e^{f_\theta(z)}\right]\right\} \\ &\leq \mathrm{TC}(x)\end{aligned}$$

$$\mathrm{D_{KL}}[q\|p] = \sup_{f:Z\to\mathbb{R}}\left\{\mathbb{E}_p[f(z)] - \log \mathbb{E}_q\left[e^{f(z)}\right]\right\}$$